

Chapter 68

How Data Hiring Works

See the full hiring process from the other side of the table, so nothing in it surprises you · write a CV that passes both the screening software and the human who reads it next · know what each interview round actually tests, for whichever of the ten roles you're aiming at

Approved chapter · v1

Contents

Chapter 68. How Data Hiring Works

Why this matters

In plain English

68.1 The whole process, gate by gate

68.2 Cracking the resume screen: the software gate, then the human gate

68.3 What each round actually tests

68.4 LinkedIn hiring: how recruiter search actually works, and how to be found

68.5 Referrals, used properly

68.6 Preparing in 30, 60, or 90 days

68.7 Worked examples, easy to complex

68.8 Four resumes, matched to their JDs

Common mistakes and how to spot them

In the real world: what Farah actually did

Tools

The project: your own gate-by-gate plan

You've got it when...

Recap

Practice exercises

Key terms

Where this leads

Chapter 68. How Data Hiring Works

Part VIII — The Interview Playbook

Chapter at a glance

You will learn to: see the full hiring process from the other side of the table, so nothing in it surprises you · write a CV that passes both the screening software and the human who reads it next · know what each interview round actually tests, for whichever of the ten roles you're aiming at · build a portfolio and a LinkedIn profile that do real work for you, not just sit there · use referrals properly, without asking for a favor you haven't earned · run a focused 30/60/90-day preparation plan instead of "studying everything."

Before you start: Chapter 7 (the ten roles and tracks) and Chapter 8, especially §8.5 (decoding a job description) and §8.6 (pay). This chapter doesn't repeat either; it builds on them.

Time needed: 2–3 hours to read, then it's a reference you return to at each stage of a real search.

Why this matters

Most candidates prepare for the *questions*. Far fewer prepare for the *process*: the sequence of filters between "I want this job" and "I have an offer," each with its own, different test. A brilliant answer to a system design question is wasted if your CV never reached a human, or if you skip the salary conversation entirely because nobody told you when it happens. This chapter is the map of that whole process, so every other chapter in this part lands where it actually matters.

In plain English

Think of hiring as a series of gates, not one big exam.

A CV goes through a gate that's often software, then a human skimming for thirty seconds, then a recruiter call, then one or more rounds that test different things, then a final conversation about money and start date. Each gate has a different judge and a different question in mind. Preparing brilliantly for gate four while walking into gate one unprepared means you never reach gate four at all. This chapter walks the gates in order.

68.1 The whole process, gate by gate

For a typical data role, in order:

1. **Application.** Your CV (and often a short form) goes in, usually through an applicant tracking system (**ATS**), software that stores, searches, and sometimes scores applications before a human sees them.
2. **Resume screen.** Either the ATS filters by keyword match, or a recruiter (sometimes both) skims for fit: right skills, right seniority, no obvious mismatch.
3. **Recruiter call** (15–20 minutes). Confirms basics: notice period, current pay expectations in general terms, location, and whether the role genuinely matches what you're looking for. This is a filter, not a technical round; treat it seriously anyway.
4. **One or more technical rounds.** SQL, Python, statistics, a case, a system design conversation, whichever mix suits the role and level (section 68.3).
5. **A hiring-manager round.** The person you'd actually report to, testing fit with how the team works, not just raw skill.
6. **Sometimes a panel or a take-home.** More common at larger companies or for senior roles (Chapter 82 covers take-homes in full).
7. **Reference checks and offer.** Pay, level, and start date are negotiated here, not before. §8.6 already covers how to read salary numbers; use that groundwork now.

Two things to notice about this list. First, **most attrition happens at gates 1 and 2**, long before anyone tests your SQL. A strong technical candidate with a CV that never gets read never gets a chance to prove it. Second, **each gate rewards different things**: gate 1 rewards keyword and format discipline; gate 4 rewards the extra-point moves from Chapter 69; gate 7 rewards knowing your numbers in advance. Prepare for the gate in front of you, not the one three steps away.

68.2 Cracking the resume screen: the software gate, then the human gate

Your CV is read twice, by two completely different readers, and it has to pass both.

How an ATS actually parses a resume

An **applicant tracking system (ATS)** is software that stores every application and, at most companies above a few hundred employees, does some combination of three things: parses your document into structured fields (name, dates, employer, skills), scores or ranks it against the JD's keywords, and lets a recruiter search and filter the stored pool. The common platforms behave slightly differently, but the failure modes are the same across all of them:

Platform (common in India-facing hiring)	Typical behavior
Workday	Strict parsing; known to struggle with non-standard date formats and merged cells
Greenhouse	Generally strong parsing; still fails on graphics-heavy templates
Lever	Similar to Greenhouse; keyword search is heavily used by recruiters on top of parsing

Platform (common in India-facing hiring)	Typical behavior
Taleo (older, still common at large enterprises)	Older parsing engine; the least forgiving of the four with tables, columns, and headers/footers
Naukri, LinkedIn, and similar job boards	Not a true ATS, but their own search and "match score" features work on the same keyword logic

What actually breaks parsing, tested the way a recruiter would test it (copy your CV's text out of the PDF and see what comes out):

- **Multi-column layouts.** A parser reading left to right across the page, not down one column then the next, can interleave your job titles with the wrong dates or your skills list with your education.
- **Text inside tables, text boxes, or headers/footers.** Many parsers skip these entirely. A contact phone number in a header, or a skills list in a table, can simply vanish from the parsed version even though it looks fine on screen.
- **Icons and images standing in for words.** A phone icon instead of the word "Phone," or a bar-chart graphic showing "Excel: Advanced," carries no text a parser can read at all.
- **Unusual date formats.** "Jan '22 – Present" parses less reliably than "January 2022 – Present"; when in doubt, spell it out.
- **Non-standard section headings.** "My Journey" instead of "Experience," or "What I Bring" instead of "Skills," can fail to map to the field the ATS expects, which means that whole section may not be scored at all.

A five-minute self-test: open your CV, select all the text, copy it, and paste it into a plain text file. If what comes out is readable, in the right order, with nothing missing, your CV will survive most ATS parsing. If it comes out scrambled, so will the parsed version a recruiter or algorithm actually scores.

Beating the keyword match, honestly

Once parsed, many systems score your CV against the JD's own language before a human ever opens it. Three concrete moves:

- **Mirror the JD's exact terms**, not synonyms, wherever they're true of you. If the JD says "stakeholder management" and you wrote "worked closely with business teams," a keyword match can miss the connection a human would instantly see. Re-read the must-have list (Chapter 8, §8.5) and match its wording directly.
- **Use both the acronym and the full term at least once**, since different systems search for different forms: "Structured Query Language (SQL)" covers a search for either.
- **Repeat your strongest 3–4 keywords naturally across two sections** (say, once in a summary line and once in the relevant bullet), which raises your match score on systems that count frequency, without ever repeating a keyword so often it reads as stuffed.

The human gate, immediately after

Assuming it survives parsing, a recruiter or hiring manager typically spends well under a minute on a first human pass. What they're scanning for, roughly in this order: right role and level, right core skills, recent and relevant experience, no obvious red flags. Structure the page so that fast scan succeeds:

- **Lead with outputs, not duties.** "Built a weekly at-risk customer list that the sales team uses to prioritize calls" beats "Responsible for customer data analysis." Chapter 9's portfolio-piece structure (§9.5: question, data, approach, result, check, decision, next step) is exactly the shape a strong CV bullet compresses into one line.
- **Quantify wherever honestly possible.** Not every number needs to be a business metric; "automated a report that took 3 hours a week" is a real, specific claim a reader can picture.
- **One page for under 5 years' experience**, two at most beyond that. Length signals judgment about what matters, not effort.
- **Projects section, for freshers and career switchers especially.** This is where your portfolio (§9.5) earns its keep on the page itself, not just behind a link.
- **A skills line near the top, not buried at the bottom.** Both readers, software and human, look there first; don't make either one hunt for it.

Watch out: don't lie to the keyword scanner. Padding a CV with skills you can't actually discuss (because you listed them purely to pass a filter) gets caught the moment a live-coding round starts. It costs you the interview you worked to reach, which is a worse outcome than not reaching it at all.

The format rules, in one place

- One column. Standard fonts (Calibri, Arial, or similar; avoid anything decorative). Standard headings: Experience, Education, Skills, Projects.
- No tables, no text boxes, no headers/footers carrying content that matters, no icons standing in for words, no photo (standard for Indian data roles; a photo can also introduce unconscious bias a recruiter would rather avoid).
- Save as the format the posting requests, usually PDF; if none is stated, PDF is the safer default, since it preserves formatting across every reader's device even though a small number of older ATS platforms parse .docx marginally more reliably. If you're applying to a company known to use an older system (word of mouth, or a careers page that looks dated), .docx is worth trying as a fallback if a PDF application seems to disappear without any response.

68.3 What each round actually tests

Match your preparation to the round in front of you, and to the role and level you're targeting from Chapter 7's ten roles.

Round	What it's really testing	How to prepare
Recruiter call	Basic fit, clear communication, realistic expectations	Know your notice period, your current CTC range (§8.6), and a one-minute answer to "tell me about yourself"
SQL / live coding	Correctness, process, how you handle being watched while you think	Chapter 71's bank; think out loud, use Chapter 69's moves

Round	What it's really testing	How to prepare
Case / business	Structured thinking under ambiguity	Chapter 75's cases; always clarify first
Statistics / ML technical	Depth of understanding, not memorized formulas	Chapters 73–74's banks; explain <i>why</i> , not just <i>what</i>
System design (senior roles)	Trade-off thinking at scale	Chapter 77's design walk-throughs
Take-home assignment	Independent work quality, code you'd actually ship	Chapter 82's worked assignments
Hiring-manager round	Team fit, how you'd actually work day to day	Specific stories, honest questions about the role
Behavioral / HR	Past behavior as a signal for future behavior	Chapter 81's STAR-method bank

Different roles weight these rounds differently. A **data analyst** interview leans on SQL, Excel/Sheets, and a business case; a **data scientist** interview adds statistics and ML depth; a **data engineer** interview adds system design and pipeline debugging; an **architect** interview is almost entirely case and system design, with far less live coding. Check the JD's must-have list (§8.5) to predict which rounds you'll actually face: a posting heavy on "dashboards" and "stakeholder communication" is unlikely to include a deep ML round, whatever the title says.

68.4 LinkedIn hiring: how recruiter search actually works, and how to be found

Most CVs are still submitted through a company's own application page, but an enormous share of data-role hiring, especially at tier 1 and tier 2, starts the other way round: a recruiter searches LinkedIn for candidates *before* a single application arrives. Understanding how that search works changes what you actually do with your profile.

How a recruiter finds you

LinkedIn Recruiter (the paid tool most companies' talent teams use, distinct from the free version you browse on) lets a recruiter filter candidates by fields such as job title, skills, location, and company, and apply Boolean logic (AND / OR / NOT, exact phrases in quotes) within those filters, mainly the Job Title, Company, and Keywords fields. A recruiter hunting for a data analyst might, for instance, set the Job Title filter to "data analyst" OR "business analyst" and add "SQL" and "Power BI" as required keywords, filtering out "intern." (LinkedIn has also begun layering AI-driven semantic search on top of this, which can match near-synonyms even without an exact keyword; Boolean keyword matching still does most of the work, and remains the safer thing to design your profile around.)

Three consequences follow directly from how that search works:

- **Your headline and "Skills" section are the highest-weighted fields.** They're what the keyword search scans first and most heavily. A headline of just "Analyst at Company X" is nearly invisible to a search for "SQL" or "Power BI," even if both appear elsewhere on your profile.

- **The exact words matter more than the truth behind them.** A recruiter searching "SQL" will not find a profile that only says "databases," exactly the same lesson as the ATS keyword match in §68.2, now applied to LinkedIn's own search instead of a company's application system.
- **Recency and activity are a filter too.** Recruiters commonly sort by "recently active" or filter to profiles updated in the last few months, on the theory that an actively maintained profile signals an actively job-seeking (or at least reachable) candidate.

Setting up a profile that gets found

- **Headline: role plus your top two or three searchable skills**, not a job title alone. "Data Analyst | SQL · Power BI · Python" is both more searchable and more informative at a glance than "Analyst at Company X."
- **Skills section: list the exact terms from JDs you're targeting**, in the same wording, not paraphrased. This is the single highest-leverage five minutes you can spend on the whole profile.
- **"Open to Work" (recruiter-only visibility mode).** This signals actively looking without alerting your current employer, and it's a filter many recruiters apply directly; leaving it off can mean a search that would have found you never surfaces you at all.
- **About section: three or four sentences**, written like the first paragraph of a portfolio piece: what you do, one concrete thing you've built, what you're looking for next.
- **Keep it current as you learn.** Chapter 9's advice to retire early portfolio pieces as better ones replace them applies here too: a stale profile from before you started this book undersells you, and (per the point above) a stale, inactive-looking profile is also less likely to surface in a search at all.

Easy Apply, InMail, and what actually gets a reply

- **"Easy Apply" postings go into the same pool as everything else**, not a lesser one, but they also draw far higher volumes; a generic, unmodified profile submitted via Easy Apply competes against hundreds of similar submissions. Treat it exactly as seriously as a company-site application: check your match against the JD (§8.5) before you click.
- **When a recruiter messages you first (InMail), reply quickly and specifically.** Response rate and response time are both tracked internally at many companies as recruiter performance metrics, which means a fast, clear reply gets genuinely more attention than the content alone might suggest. A reply that just says "Interested!" wastes the opening; a reply that references one specific duty from their message performs far better ("Thanks for reaching out, the automation piece you mentioned is exactly what I've been building toward").
- **A profile photo and a completed profile both measurably raise search ranking and response rates** on LinkedIn's own stated logic (a completed profile signals a real, active user); this is a rare case where a small effort has an outsized, well-documented payoff.

This changes; verify before you rely on it. LinkedIn's search mechanics, filters, and Easy Apply behavior are updated by the platform regularly, and hiring practices shift with them. The mechanics above were checked in September 2026; search LinkedIn's own current Recruiter help pages, or a recent, reputable sourcing guide, before treating any specific detail here as current.

Portfolio, alongside LinkedIn

Chapter 9, §9.5 already covers how to *build* a portfolio piece. Two rules for using it inside the hiring process itself:

- **Link it from both your CV and your LinkedIn profile's "Featured" section**, not just a GitHub profile page. A reader on either surface shouldn't have to hunt.
 - **Pick your best two or three pieces for a given application**, not all of them. A recruiter who opens a portfolio wants a quick, relevant look, not an archive; reference the most relevant piece by name in a cover note or application answer where it fits: "In the project linked below, I built an at-risk customer list on a similar problem to what this role's JD describes."
-

68.5 Referrals, used properly

A referral doesn't skip the technical bar; it skips the resume-screen gate, which, per section 68.1, is where most candidates are lost before anyone tests their skills. That's still a large, legitimate advantage, worth pursuing correctly.

- **Ask for a referral only after you've decoded the JD (§8.5) and know you clear most of the must-haves.** Asking a connection to refer you into a role you're not close to ready for spends their credibility, not just your time.
 - **Make it easy for them.** Send your CV, the job link, and two or three sentences on why you're a fit, the same duties-to-outputs mapping §8.5 teaches, so they can forward it in under a minute.
 - **Build the relationship before you need it.** A message asking a near-stranger for a referral the same day lands very differently from one to someone whose posts you've engaged with, or who you've had one real conversation with previously.
 - **Thank them regardless of outcome**, and tell them what happened. People refer again for candidates who close the loop (Chapter 69's move 12, applied to your own professional relationships).
-

68.6 Preparing in 30, 60, or 90 days

How much runway you have changes what "prepared" should mean. Farah Khan, the Riverstone sales executive from Chapters 8 and 9 who's working toward a data analyst role, is a useful case: she scored 5 of 10 on the must-haves in Chapter 8's JD (§8.5): strong on spreadsheets, communication, and CRM experience; gaps in SQL, a BI tool, and a shipped automation. Here's how her plan would differ by runway.

90 days: close the real gaps

Farah has time to genuinely close her three gaps, not just talk about them: SQL to a working level (Chapters 12–13), a BI tool (Chapter 16), and one real automated report she can show (Chapters 19–20). With 90 days, the right order is **skills first, mock interviews last two weeks**: practicing interview

technique on skills you don't have yet wastes the technique.

- **Weeks 1–8:** the three skill gaps, following this book's own chapter order, each ending in a portfolio piece (§9.5).
- **Weeks 9–11:** Chapters 69–71 (and 76, if targeting BA-flavored roles too), drilling the extra-point moves on real questions.
- **Week 12:** mock interviews (Chapter 82), applications, and referral outreach (68.5) begin in parallel with the drilling, not after it.

60 days: one deep gap, plus interview technique

With 60 days, Farah can close her single biggest gap (SQL) properly and get her BI tool and automation to "can demonstrate, if not expert" rather than "can do independently." Interview preparation starts earlier, around week 5, running alongside the remaining skill-building rather than strictly after it.

30 days: triage, don't spread thin

Thirty days isn't enough to close three genuine skill gaps from zero. The honest move is **triage**: pick the one gap that blocks the most roles (usually SQL, per §7.2's "the one skill every role shares"), get it to a defensible level, and lean hard on Chapter 69's method to make existing strengths (spreadsheets, communication, CRM experience, for Farah) score as high as they possibly can. A 30-day plan that tries to touch everything shallowly loses to one that closes a single real gap and interviews confidently on the rest.

Interview extra point. Whatever your runway, bring your *decoded* job description into the interview itself (§8.5's method). When asked "why this role," name two specific duties from the posting and say exactly what you've built that matches each, then ask one honest question about the hidden skill the JD reveals ("How are definitions like 'active customer' agreed today?"). It signals you read the posting as a document to understand, not a form to pattern-match against, precisely the "structure and business judgment" dimensions Chapter 69's rubric rewards, applied to the conversation about the job itself, not just its technical content.

68.7 Worked examples, easy to complex

Ten examples, graded from easy to hard, with full solutions. Each pulls together the sections above; read them in order even if you only need one, the reasoning builds.

Example 1 (Easy): CV bullet, a fresher's single duty

Situation: an unpaid college project, one task: cleaning a messy public dataset.

Weak bullet: "Worked on data cleaning for a college project using Python."

Solution: apply 68.2's "output, not duty" rule. What was the question, what was the result, what would a reader picture?

"Cleaned a 40,000-row public transport dataset (missing values, inconsistent city names) in pandas, cutting duplicate records from 12% to under 1%."

Why it's stronger: a number (40,000 rows, 12% to under 1%) replaces a vague verb ("worked on"), and it names the specific problem (missing values, inconsistent names) a reader in the field recognizes immediately.

Example 2 (Easy): CV bullet, translating a non-data job

Situation: two years as a store manager, no formal "data" title, moving into a BA or analyst track.

Weak bullet: "Managed daily store operations and staff."

Solution: find the data-shaped work hiding inside a non-data title. A store manager tracks stock, staffing, and sales numbers daily: that's reporting and process work by another name.

"Built a weekly Excel stock-and-sales tracker across 3 store locations, used by regional management to reallocate inventory; cut stockouts by an estimated 15%."

Why it's stronger: this is Example 1's rule applied to a harder case: the underlying achievement existed all along, it just needed translating from "manager" language into "output" language. This is exactly the translation §8.5 teaches for reading a JD, run in reverse, on your own history.

Example 3 (Medium): Scoring a Business Analyst JD

Situation: a posting for "Business Analyst, Operations," tier 1, requiring: process mapping, stakeholder interviews, SQL (basic), Excel (advanced), a ticketing tool (Jira preferred); nice-to-have: BPMN certification, exposure to Power BI.

Candidate: two years as a coordinator, no formal BA title. Self-scores (0/1/2, per §8.5): process mapping 1 (done informally, no training), stakeholder interviews 2 (ran vendor meetings weekly), SQL 0, Excel 2, ticketing tool 1 (used Trello, not Jira), BPMN 0, Power BI 0.

Solution:

Skill	Type	Score	Weight	Points
Process mapping	Must	1	2	2
Stakeholder interviews	Must	2	2	4
SQL (basic)	Must	0	2	0
Excel (advanced)	Must	2	2	4
Ticketing tool	Must	1	2	2
BPMN	Nice	0	1	0
Power BI	Nice	0	1	0
Total				12 / 26

Must-have coverage: $(1+2+0+2+1) / 10 = 60\%$.

Reading it. Worth applying: 60% must-have coverage with one clean gap (SQL) and one near-miss (Trello, not Jira, which is usually close enough to mention honestly and offer to learn). The two zero-scores on nice-to-haves don't matter; §8.5 is explicit that nice-to-haves are tie-breakers, not requirements.

Example 4 (Medium): Scoring a Data Scientist JD with an inflated tool list

Situation: "Data Scientist," must-haves: Python, statistics, SQL, "machine learning," and, unusually for a tier 2 posting, TensorFlow, Docker, Kubernetes, and Airflow all listed as required, not nice-to-have.

Solution: before scoring, apply §8.5's red-flag check: "a tool list longer than the duties" for a role whose actual duties (read further down the posting) are "build models to predict customer churn and present results to stakeholders." Four infrastructure tools as hard requirements for a role that's really about modeling and communication is a mismatch, likely a copy-pasted posting rather than a true requirement list.

Practical scoring move: score the two tiers separately. Core (Python, statistics, SQL, ML): score normally. Infrastructure tools (TensorFlow, Docker, Kubernetes, Airflow): treat as a soft signal, not a hard gate, and raise it directly in the recruiter call ("the posting lists Kubernetes as required: is that truly day-one, or does the team support ramping up on it?"). This is Move 1 from Chapter 69 (clarify first), applied to the application stage instead of an interview answer.

Reading it. A rigid score against the literal JD would wrongly disqualify a strong-fit candidate. The skill here isn't scoring faster; it's recognizing when the JD itself needs a clarifying question before the score means anything.

Example 5 (Hard): Scoring for a senior/architect role, with judgment calls

Situation: "Data Architect," 8+ years, must-haves include "proven experience designing systems for 10M+ daily events," "stakeholder alignment across engineering and business," and a named cloud platform.

Candidate: 6 years' experience, designed a system handling roughly 2M daily events, strong stakeholder track record, experience on a different cloud platform than the one named.

Solution: senior JDs reward honest judgment over literal score-counting. Three specific calls:

1. **"8+ years" against 6.** Chapter 8's own guidance treats years as a guide, not a hard wall, especially when depth compensates for tenure; score it 1, not 0, and address the gap directly if asked, rather than self-disqualifying before applying.
2. **"10M+ daily events" against 2M.** This is a genuine, material gap for an architect role specifically about scale; score it honestly at 0–1, and prepare a system-design answer (Chapter 77) that shows you understand *how* the design would change at 5x the scale, even without having run it.
3. **A different cloud platform.** Score close to full credit; core architectural thinking (Chapter 77's design walk-throughs) transfers across platforms far more than junior-level tool skills do, and most architect interviews test the thinking, not platform trivia.

Reading it. At senior levels, the scoring method from §8.5 still applies, but its main value shifts from "should I apply" (almost always yes, if two of three must-haves are close) to "which specific gap do I need a strong, rehearsed answer for in the interview."

Example 6 (Easy): Referral message, a warm connection

Situation: a former teammate now works at the target company; you've spoken in the last month.

Solution:

"Hi Farhan, hope the new role's going well! I saw Riverstone Traders is hiring a Data Analyst (link) and it's a strong match for what I've been building toward, especially the automation piece. I've attached my CV, would you be open to referring me, or pointing me to whoever's hiring for it?"

Why it works: short, names the specific role, ties one concrete strength to one concrete duty (per 68.5), and asks a clear, low-effort question.

Example 7 (Medium): Referral message, a cold LinkedIn connection

Situation: no prior relationship; connecting with someone at the target company specifically to ask for a referral.

Weak version: "Hi, I saw you work at X. I'm looking for a job, could you refer me?" (No context, asks a stranger for a favor in the first message.)

Solution: two messages, not one.

Message 1 (connect, no ask): "Hi Priya, I've been following Riverstone's work in [specific area] and would love to connect as I explore roles in data analytics."

Message 2, a few days later, only after they accept: "Thanks for connecting! I noticed the Data Analyst opening on your team (link). I've spent the last few months building SQL and Power BI skills specifically around sales analytics, including a project on at-risk customer scoring. Would you be open to a referral, or a quick pointer to who's hiring for it?"

Why it works: it earns the ask across two messages instead of spending a stranger's goodwill in the first line, exactly 68.5's "build the relationship before you need it" rule, compressed into the shortest version that still respects it.

Example 8 (Hard): Re-approaching after a past rejection, for a different role

Situation: rejected for a Data Analyst role at a company six months ago; a new Analytics Engineer posting has opened, and you've closed your dbt/SQL gap since then.

Solution:

"Hi Vikram, we spoke a few months back about the Data Analyst role. I understand the fit wasn't quite right then. Since then I've focused specifically on SQL and dbt (project link), and I noticed the Analytics Engineer opening, which maps closely to that work. Would it be worth a quick conversation, or should I apply directly?"

Why it works: it names the past interaction honestly instead of pretending it didn't happen (which the interviewer will remember anyway), states exactly what changed, and offers the reader an easy way to redirect you if the timing still isn't right. This is Chapter 69's Move 10 (admit limits honestly) applied to your own hiring history, not a technical answer.

Example 9 (Medium): A 90-day plan for a true fresher

Situation: unlike Farah Khan (68.6), a recent graduate with coursework but no work-relevant portfolio at all, and no existing professional network to draw referrals from.

Solution: the shape from 68.6 still applies, but week 1 needs an extra step Farah didn't: choosing *which* track (Chapter 7, §7.2) to aim at, since a true fresher hasn't narrowed it the way someone already working toward a specific role has.

- **Week 1:** pick one track using Chapter 7's map; resist the urge to prepare for all ten roles at once.
- **Weeks 2–9:** identical shape to Farah's skills-first weeks, but built from zero rather than from an existing job's worth of transferable experience; expect this to fill the full eight weeks rather than leave slack.
- **Weeks 8–10 (overlapping, since there's no existing network to lean on early):** LinkedIn and portfolio go live in week 8, not held until "ready," since a fresher's biggest gate-1 risk is having nothing to link at all; referral outreach starts cold (Example 7's two-message pattern) rather than warm, since Example 6's shortcut isn't available yet.

Reading it. The plan's shape doesn't change with experience level; what changes is which weeks get compressed and which need the extra step this example adds.

Example 10 (Hard): A 60-day plan for a lateral move, not a beginner

Situation: an experienced Business Analyst (4 years) moving toward Data Scientist, with real SQL and stakeholder skills already, but no ML background at all.

Solution: this is a harder planning problem than either Farah's or Example 9's, because the candidate isn't weak, they're **misfiled**: strong on one track's skills, applying to a different track's roles. Two adjustments to 68.6's method:

1. **Don't re-learn what already transfers.** SQL and stakeholder communication (Chapter 7's BA row) map almost directly onto a DS role's needs; scoring them at 0 out of false modesty wastes runway that should go to the real gap.
2. **The real gap (statistics and ML, Chapters 21–22, 35–39) is deep, not shallow**, so 60 days is tight. The honest move, per 68.6's 60-day guidance, is triage within the gap itself: prioritize the statistics and evaluation chapters (heavily tested in every DS interview, Chapter 73) over the deep-learning material (Chapter 43), which is tested far less often at this level and can be learned after an offer, not before it.

Reading it. A lateral move from an adjacent track is not "starting over"; treating it that way wastes the runway's most valuable resource, which is the skills you don't have to rebuild.

68.8 Four resumes, matched to their JDs

Four complete resumes, each paired with a condensed job description it was actually built to clear, across four different roles and levels. Read each JD first, then the resume, then the annotation, which points out exactly why the resume clears both the software gate and the human gate from §68.2.

68.8.1 Fresher, Data Analyst

JD excerpt: Data Analyst (Fresher), TrueNorth Retail Pvt. Ltd.

0–1 years' experience; freshers with strong projects considered. Must have: SQL (joins, aggregation), Excel or Google Sheets (pivot tables, lookups), basic data visualization. Nice to have: Python, a BI tool.

Resume

Aditi Sharma Pune, Maharashtra · aditi.sharma.data@email.com · +91-98XXXXXXX1 · linkedin.com/in/aditisharma-data · github.com/aditisharma-data

Summary Data Analyst fresher with hands-on SQL, Excel, and Power BI project experience on realistic business datasets. Comfortable turning a business question into a clean, checked answer.

Skills SQL (joins, aggregation, window functions) · Excel and Google Sheets (pivot tables, lookups) · Power BI · Python (pandas, basic)

Projects

- **At-risk customer list, retail dataset (SQL, Power BI).** Built a query identifying customers with no order in 90+ days, ranked by revenue; visualized in a Power BI dashboard used to prioritize sales follow-up. Validated totals reconciled to source data before presenting.
- **Sales data cleaning and analysis (Python, pandas).** Cleaned a 12,000-row dataset (missing values, duplicate records), then analyzed monthly revenue trends by category; found and flagged a data entry error affecting 3% of rows.
- **Automated weekly report (Google Sheets, Apps Script).** Replaced a 45-minute manual weekly task with a script triggered automatically every Monday morning.

Education B.Com, Savitribai Phule Pune University, 2022–2025

Certifications Google Data Analytics Professional Certificate, 2025

Why this clears both gates. Software gate: single column, standard headings, every must-have keyword from the JD (SQL, Excel, pivot tables, basic visualization) appears in the Skills line in the JD's own words. Human gate: no prior job title to lean on, so every project bullet follows §68.2's "output, not duty" rule with a number attached, and the project section does the work a fresher's experience section can't.

68.8.2 Career switcher, Business Analyst

JD excerpt: Business Analyst, Operations, Meridian Logistics

2+ years in any operations, coordination, or analyst role. Must have: process mapping, stakeholder communication, SQL (basic), Excel (advanced). Nice to have: Jira or similar ticketing tool, BPMN exposure.

Resume

Rohan Verma Bengaluru, Karnataka · rohan.verma.ba@email.com · +91-98XXXXXXX2 · linkedin.com/in/rohanverma-ba

Summary Operations professional moving into business analysis, with 3 years managing supplier coordination and reporting for a multi-location retail operation. Strong stakeholder communication; building SQL and process-mapping skills alongside hands-on project work.

Skills Process mapping · Stakeholder communication · Excel (advanced: pivot tables, lookups, what-if analysis) · SQL (joins, aggregation) · Trello (ticketing)

Experience **Operations Coordinator, Bluepeak Retail Pvt. Ltd.** | Bengaluru | 2022–Present

- Built a weekly Excel stock-and-sales tracker across 3 store locations, used by regional management to reallocate inventory; contributed to an estimated 15% reduction in stockouts.
- Ran weekly vendor coordination meetings across 8 suppliers, resolving delivery and quality issues; documented recurring issues into a process map that cut average resolution time by 2 days.
- Coordinated a system migration between two teams, translating operational needs into requirements for the technical team.

Projects

- **Requirements and process map, order fulfillment workflow (SQL, process mapping).** Documented the current order-to-delivery process end to end, identified 3 redundant approval steps, and wrote a query analyzing where orders were delayed most often.

Education BBA, Christ University, 2018–2021

Certifications CBAP coursework in progress, 2026

Why this clears both gates. Software gate: "process mapping" and "stakeholder communication," the JD's own phrases, appear directly rather than as the paraphrase ("worked closely with business teams") a switcher's first instinct often produces. Human gate: the Experience section translates operations duties into outputs using exactly Example 2's method from §68.7, and the one Projects entry signals BA-specific intent (a formal process map and a query) beyond what the day job alone shows.

68.8.3 Mid-level, Data Scientist

JD excerpt: Data Scientist, 3–5 years, Northbridge Analytics

Must have: Python, statistics and hypothesis testing, SQL, machine learning (supervised learning, model evaluation). Nice to have: A/B testing experience, cloud platform exposure (any).

Resume

Kavya Nair Hyderabad, Telangana · kavya.nair.ds@email.com · +91-98XXXXXXX3 · linkedin.com/in/kavyanair-ds · github.com/kavyanair-ds

Summary Data Scientist with 4 years building and evaluating models for customer retention and demand forecasting, from business question through to a monitored result. Strong on honest evaluation, not just model accuracy.

Skills Python (pandas, scikit-learn, statsmodels) · SQL · Statistics and hypothesis testing · Machine learning (logistic regression, gradient boosting, evaluation and calibration) · A/B testing · AWS (S3, basic)

Experience Data Scientist, Solvix Technologies | Hyderabad | 2023–Present

- Built a churn prediction model (gradient boosting) reaching 0.79 AUC, evaluated against a cost-based threshold rather than accuracy alone; the resulting call list protected an estimated ₹1.1M more revenue than a probability-only ranking over 6 months.
- Designed and analyzed an A/B test on a pricing change, correctly accounting for a stopping-time bias the initial design had missed before launch.
- Explained model outputs to non-technical stakeholders using SHAP-based summaries, translating "why did this account score high risk" into specific, actionable account notes.

Data Analyst, Kestrel Retail | Hyderabad | 2021–2023

- Built the SQL and Python pipeline behind a weekly demand forecast, cutting stockouts by 18% over the following two quarters.

Projects

- **Open-source demand forecasting comparison (Python, statsmodels, Prophet).** Backtested five forecasting methods on multi-year retail data; found the simplest method (seasonal naive) beat two more complex ones on the noisiest series, and wrote up why.

Education B.Tech, Computer Science, VNIT Nagpur, 2017–2021

Certifications None currently listed (deliberately: at this level, shipped project results carry more weight than certificates, and the space is better spent on them)

Why this clears both gates. Software gate: every must-have term (statistics, hypothesis testing, SQL, machine learning, model evaluation) appears in the Skills line in the JD's own words, with A/B testing and a cloud platform (the nice-to-haves) also present rather than omitted. Human gate: both

experience bullets and the one project follow Chapter 9's full seven-part portfolio shape compressed into one line each (question, approach, result, and a check or honest caveat), which is exactly what separates a mid-level DS resume that gets a callback from one that reads as a list of tool names.

68.8.4 Mid-level, Data Engineer

JD excerpt: Data Engineer, 3+ years, Fernhill Systems

Must have: SQL, Python, pipeline orchestration (Airflow or similar), cloud data warehouse (any major platform). Nice to have: dbt, streaming/real-time experience, data quality frameworks.

Resume

Arjun Reddy Chennai, Tamil Nadu · arjun.reddy.de@email.com · +91-98XXXXXXX4 · linkedin.com/in/arjunreddy-de · github.com/arjunreddy-de

Summary Data Engineer with 3 years building and maintaining pipelines feeding a company's core reporting and ML systems, with a focus on pipelines that fail loudly and get fixed fast, not silently.

Skills SQL · Python · Apache Airflow · Snowflake · dbt · Data quality testing (Great Expectations) · Basic streaming (Kafka, exposure)

Experience Data Engineer, Vantara Systems | Chennai | 2023–Present

- Rebuilt a daily sales pipeline (Airflow, Snowflake, dbt) that previously failed silently roughly once a month; added automated data-quality checks and alerting, cutting undetected failures to zero over the following year.
- Migrated 40+ legacy SQL reports into tested dbt models with documentation, reducing a recurring "which number is right" dispute between two teams to effectively none.
- On-call rotation for pipeline incidents; wrote the team's first incident runbook after a recurring late-data issue, cutting average resolution time from 3 hours to 40 minutes.

Data Analyst → Junior Data Engineer, Kestrel Retail | Chennai | 2021–2023

- Started as a data analyst; moved into engineering after automating a reporting pipeline that then became the team's standard approach for two other reports.

Projects

- **Personal streaming pipeline (Python, Kafka, Docker).** Built a small real-time pipeline ingesting a public API, to learn streaming concepts not used day-to-day at work; documented the design trade-offs against the batch approach used at the job.

Education B.E., Information Technology, Anna University, 2017–2021

Why this clears both gates. Software gate: the must-have list (SQL, Python, an orchestration tool, a cloud warehouse) and two nice-to-haves (dbt, a data quality framework) all appear in the exact JD wording. Human gate: the bullets emphasize reliability and incident response, not just pipelines built,

which is precisely what a data engineering hiring manager's fast scan is checking for beyond the tool list, and the one side project shows initiative on a specific named gap (real-time/streaming) rather than a generic "I like building things" claim.

Common mistakes and how to spot them

Mistake	Symptom	Fix
A CV formatted for humans, not software	Never hears back, even from roles you're qualified for	Standard single-column format; match the JD's exact keywords
Preparing only for technical rounds	Never reaches them, because the CV or recruiter call fails first	Prepare gate 1 and gate 3 as seriously as gate 4
A portfolio with no write-up	Reviewer can't tell what you actually did or decided	Use §9.5's seven-part structure; lead with the question and the decision
Asking for a referral cold, underqualified	Damages the relationship, rarely works anyway	Decode the JD first; only ask once you clear most must-haves
A 30-day plan that tries to cover everything	Shallow on every skill, confident on none	Triage to the one gap that blocks the most roles
Treating the recruiter call as a formality	Loses the role on basics before any technical round	Know your numbers and your one-minute pitch cold

In the real world: what Farah actually did

Farah's must-have coverage from Chapter 8 was 50%: strong on spreadsheets, communication, and CRM experience; gaps on SQL, a BI tool, and a shipped automation. She had roughly ten weeks before the role she wanted was likely to close, which put her plan closer to the 90-day shape, compressed.

She didn't apply immediately. She spent the first six weeks on SQL and Power BI, building two portfolio pieces along the way: a cleaned dataset with a documented quality issue, and a small at-risk-customer query she could explain end to end, including one deliberate edge case she'd found and fixed. She updated her LinkedIn headline the same week she finished the SQL chapters, not the week she felt "ready," on the theory that recruiters search continuously and a stale profile costs her nothing to fix early.

By week eight, her must-have coverage, honestly re-scored the same way, had risen to about 80%. She asked a former colleague, already at a company she was targeting, for a referral, sending the JD, her CV, and two sentences mapping her strongest project to the role's top duty. The referral moved her straight past the resume screen, but the technical round still tested exactly what it would have tested anyway, and she still needed Chapter 69's moves to turn a correct SQL answer into a strong one live. The referral bought her a seat at the table. It didn't do the sitting for her.

Tools

No software beyond what's already in your hands: the JD-scoring method from §8.5, the portfolio structure from §9.5, and whichever ATS your target companies use (worth a quick search for "how [Company]'s application system works," since a few, Workday and similar systems especially, have known formatting quirks worth checking before you apply).

The project: your own gate-by-gate plan

Goal: a written plan you'd actually follow, not a vague intention to "get ready."

Steps:

1. Find one real job posting for a role you want and score your fit against it using §8.5's method.
 2. Identify your one or two biggest gaps, and your two or three strongest existing matches.
 3. Decide your realistic runway: 30, 60, or 90 days, honestly.
 4. Write a week-by-week plan using this chapter's shape for that runway, naming which chapters of this book close which gap.
 5. Update your CV and LinkedIn headline now, this week, against the JD's actual language, whether or not you feel ready to apply yet.
-

You've got it when...

- ☐ I can name every gate in the process, in order, and what each one actually tests.
 - ☐ My CV is in a format that survives an ATS, using the target JD's own keywords honestly.
 - ☐ I know which interview rounds to expect for my target role, and I'm not spending equal effort on all of them.
 - ☐ My portfolio and LinkedIn profile are current, linked from my CV, and structured around outputs, not duties.
 - ☐ I only ask for a referral once I've honestly scored myself against the JD.
 - ☐ I have a written, runway-appropriate plan, not just a study list.
-

Recap

- Hiring is a **sequence of gates**, each judged differently; most candidates are lost at the CV and recruiter-call gates, long before any technical round.
- A CV has to pass **software first, a human second**; format and keyword-matching decide whether a human ever sees it.

- **Different rounds test different things**, and different roles weight rounds differently; predict your rounds from the JD's must-have list.
 - A **portfolio** (§9.5) and a current **LinkedIn profile** do real work in the process, not just decoration.
 - A **referral** skips the resume screen, not the technical bar; ask only once you've honestly scored yourself against the JD.
 - **30/60/90-day plans** aren't the same plan run for different lengths of time: shorter runway means real triage to the one gap that blocks the most roles, not shallow coverage of everything.
-

Practice exercises

1. Find a real job posting for a role on your track (Chapter 7, §7.2) and score your must-have coverage using §8.5's method.
 2. List the three interview rounds you'd most likely face for that posting, and say why, based on its must-have list.
 3. Write your CV's top bullet for your strongest project using the "output, not duty" rule from 68.2.
 4. Draft the two-sentence referral request you'd actually send for that posting, once your must-have coverage clears 70%.
 5. Write your own 30-, 60-, and 90-day plan for the same posting, and say which one matches your real, honest runway.
-

Key terms

applicant tracking system (ATS) · resume screen · keyword match · recruiter call · hiring-manager round · take-home assignment · referral · must-have coverage · portfolio piece · runway · triage (in preparation)

Where this leads

- **Chapter 69, The Extra-Points Method**, is what you use once you're through the gates and into a live round.
- **Chapters 70–82** are the question banks for each specific round this chapter maps out.
- **Chapter 81, Behavioral, HR & Offer Conversations**, picks up exactly where this chapter's gate 7 leaves off: the offer conversation itself.
- **Chapter 82, Take-Home Assignments & Mock Interviews**, is where the 90-day plan's final weeks, and this chapter's whole gate sequence, get rehearsed end to end.